

1. 5-line beginner summary

Machine Learning is a way to teach computers to learn patterns from data instead of writing fixed rules manually. In supervised learning, the model learns from examples where the correct answer is already known. In unsupervised learning, the model finds hidden patterns or groups without correct answers being provided. A good ML model should perform well on new unseen data, not only on training data. In real projects, ML moves from notebook experiments to production systems through testing, deployment, monitoring, and retraining.

Day 2: Machine Learning Fundamentals

2. What Machine Learning is

Machine Learning is a branch of Artificial Intelligence where computers learn from data.

Instead of writing rules like this:

```
IF customer_age > 60 AND income < 30000 THEN high_risk
```

we give the computer historical data and let it learn the pattern.

Example:

Customer Age	Income	Loan Repaid?
25	50000	Yes
45	90000	Yes
62	25000	No
35	30000	No

The ML model studies this data and learns:

People with certain income, age, credit history, and past behavior may be more or less likely to repay a loan.

So, Machine Learning is about learning patterns from past data and using those patterns to make predictions on new data.

3. Main types of Machine Learning

Simple comparison table

Type	Learns From	Output	Example
Supervised Learning	Data with correct answers	Prediction	Predict loan approval
Unsupervised Learning	Data without correct answers	Groups or patterns	Group customers by behavior

Type	Learns From	Output	Example
Classification	Labeled data	Category/class	Spam or not spam
Regression	Labeled data	Number	Predict house price
Clustering	Unlabeled data	Groups	Customer segments

4. Supervised Learning

Supervised learning means the model learns from data where the answer is already available.

The answer column is called the **target variable** or **label**.

Example:

Email Text	Label
"Win lottery now"	Spam
"Meeting at 3 PM"	Not Spam
"Free coupon claim now"	Spam

The model learns from these examples and later predicts whether a new email is spam or not.

Practical examples

Problem	Input Data	Target
Predict employee attrition	Age, role, salary, experience	Leave / Stay
Predict house price	Location, size, rooms	Price
Predict fraud	Transaction amount, location, user history	Fraud / Not Fraud
Predict customer churn	Usage, complaints, plan type	Churn / Not Churn

Supervised learning is mainly used for:

Classification Regression

5. Unsupervised Learning

Unsupervised learning means the model gets data but no correct answer column.

The model tries to find patterns, groups, or structure on its own.

Example:

Customer	Monthly Spend	Website Visits	Support Tickets
A	5000	20	1
B	300	2	0
C	4500	18	2
D	250	1	0

The model may discover:

Group 1: High-value active customers
Group 2: Low-spend inactive customers

This is useful when we do not know the labels in advance.

Common unsupervised learning use cases:

Use Case	Example
Customer segmentation	Group customers by buying behavior
Anomaly detection	Find unusual transactions
Topic grouping	Group documents by similar themes
Recommendation systems	Find similar users or products

6. Classification

Classification is used when the output is a category.

The model predicts one class from a fixed set of classes.

Examples:

Problem	Possible Output
Email spam detection	Spam / Not Spam
Loan approval	Approved / Rejected
Sentiment analysis	Positive / Negative / Neutral
Disease prediction	Disease / No Disease
Resume screening	Shortlist / Reject

Binary classification

Only two classes.

Example:

Fraud / Not Fraud
Churn / Not Churn
Yes / No

Multi-class classification

More than two classes.

Example:

```
Low Risk / Medium Risk / High Risk  
Positive / Neutral / Negative  
Cat / Dog / Bird
```

Simple example

Suppose we want to predict whether a customer will leave a telecom company.

Input features:

```
monthly_bill  
number_of_complaints  
contract_type  
internet_usage  
customer_age
```

Target:

```
churn = Yes or No
```

The model learns patterns like:

```
Customers with high complaints and month-to-month contracts may have higher  
churn risk.
```

7. Regression

Regression is used when the output is a number.

Examples:

Problem	Output
Predict house price	₹80,00,000
Predict salary	₹18,00,000
Predict delivery time	35 minutes
Predict sales	₹5,00,000
Predict temperature	32.5°C

Regression is useful when the answer is continuous.

Simple example

Problem:

```
Predict house price.
```

Input features:

```
area_sqft  
number_of_bedrooms  
location  
age_of_property  
parking_available
```

Target:

```
house_price
```

The model learns patterns like:

```
Larger houses in better locations usually have higher prices.
```

8. Clustering

Clustering is an unsupervised learning technique.

It groups similar data points together.

There is no target column.

Example:

A bank wants to group customers based on behavior.

Input features:

```
monthly_spend  
number_of_transactions  
loan_amount  
credit_card_usage  
savings_balance
```

The clustering model may create groups like:

Cluster	Meaning
Cluster 1	High-income premium customers
Cluster 2	Low-usage customers
Cluster 3	Heavy credit card users
Cluster 4	Loan-focused customers

The model does not know the names of these groups. Humans analyze the groups and give business meaning to them.

Common clustering algorithms:

Algorithm	Simple Meaning
K-Means	Creates K groups based on similarity

Algorithm	Simple Meaning
Hierarchical Clustering	Builds tree-like groups
DBSCAN	Finds dense groups and outliers

9. Feature Engineering

Feature engineering means creating useful input variables for the ML model.

Raw data is often not directly useful. We transform it into meaningful features.

Example raw data:

Customer	Date Joined	Last Login Date	Total Orders
A	2021-01-01	2024-06-01	50

Useful engineered features:

```
customer_tenure_days
days_since_last_login
average_orders_per_month
is_active_customer
```

Examples of feature engineering

Raw Data	Engineered Feature
Date of birth	Age
Transaction date	Day of week, month, weekend flag
Text review	Word count, sentiment score
Address	City, state, region
Purchase history	Average order value
Login history	Days since last login

Why feature engineering matters

A model is only as good as the information we give it.

Bad feature:

```
Customer ID
```

Usually this does not help because it is just an identifier.

Good feature:

```
Average monthly spend
```

This helps because it describes behavior.

In GenAI/RAG context

Feature engineering is also useful in AI/GenAI systems.

Examples:

GenAI Use Case	Feature-like Input
RAG system	Document chunk size, metadata, embeddings
Search ranking	Query similarity score
LLM monitoring	Prompt length, response time, feedback score
Document classification	Text length, keywords, embeddings

10. Train/Test Split

Train/test split means dividing data into two parts:

Training data: used to teach the model
Testing data: used to check model performance on unseen data

Example:

If we have 10,000 records:

80% training data = 8,000 records
20% testing data = 2,000 records

The model learns only from the training data. Then we test it on the test data.

This helps us answer:

Can the model perform well on new data it has not seen before?

Simple diagram

```
Full Dataset
|
|----> Training Data
|           |
|           ---> Model learns patterns
|
|----> Test Data
|           |
|           ---> Model performance is checked
```

Common split ratios

Split	Meaning
70/30	70% train, 30% test
80/20	80% train, 20% test

Split	Meaning
90/10	Used when dataset is large

For small datasets, cross-validation is usually better.

11. Cross-Validation

Cross-validation is a better way to evaluate a model.

Instead of using only one train/test split, we split data multiple times.

The most common method is **K-Fold Cross-Validation**.

If K = 5:

Data is divided into 5 parts.
Model trains 5 times.
Each time, one part is used for validation and remaining parts for training.

ASCII example

```
Fold 1: [Test]  [Train] [Train] [Train] [Train]
Fold 2: [Train] [Test]  [Train] [Train] [Train]
Fold 3: [Train] [Train] [Test]  [Train] [Train]
Fold 4: [Train] [Train] [Train] [Test]  [Train]
Fold 5: [Train] [Train] [Train] [Train] [Test]
```

Final score:

Average of all 5 validation scores

Why cross-validation is useful

It gives a more reliable estimate of model performance.

A single train/test split can be lucky or unlucky.

Cross-validation reduces that risk.

12. Model Evaluation Metrics

Model evaluation means checking how good the model is.

Different problems need different metrics.

Metric selection table

Problem Type	Common Metrics
Classification	Accuracy, Precision, Recall, F1-score, ROC-AUC
Regression	MAE, MSE, RMSE, R ²
Clustering	Silhouette Score, Davies-Bouldin Score

13. Classification Metrics

To understand classification metrics, first understand the confusion matrix.

Confusion Matrix

Example: Fraud detection.

Actual / Predicted	Predicted Fraud	Predicted Not Fraud
Actual Fraud	True Positive	False Negative
Actual Not Fraud	False Positive	True Negative

Meaning

Term	Meaning
True Positive	Model correctly predicted positive
True Negative	Model correctly predicted negative
False Positive	Model wrongly predicted positive
False Negative	Model wrongly predicted negative

Example:

True Positive: Actual fraud and model predicted fraud.
False Positive: Actual normal transaction but model predicted fraud.
False Negative: Actual fraud but model predicted normal.

False negatives are very dangerous in fraud detection because fraud is missed.

Accuracy

Accuracy tells how many predictions were correct overall.

$\text{Accuracy} = \text{Correct Predictions} / \text{Total Predictions}$

Example:

Out of 100 emails, model predicted 90 correctly. Accuracy = 90%
--

Accuracy is useful when classes are balanced.

But it can be misleading when classes are imbalanced.

Example:

990 normal transactions
10 fraud transactions

If the model predicts every transaction as normal:

Accuracy = $990 / 1000 = 99\%$

But the model caught zero frauds.

So accuracy alone is not enough.

Precision

Precision answers:

Out of all cases predicted as positive, how many were actually positive?

Formula:

Precision = $\text{True Positive} / (\text{True Positive} + \text{False Positive})$

Example:

Model predicted 20 transactions as fraud.
Out of those, 15 were actually fraud.
Precision = $15 / 20 = 75\%$

High precision means fewer false alarms.

Important when false positives are costly.

Example:

If a model wrongly blocks genuine bank transactions, customers may get frustrated.

Recall

Recall answers:

Out of all actual positive cases, how many did the model correctly find?

Formula:

Recall = $\text{True Positive} / (\text{True Positive} + \text{False Negative})$

Example:

There were 30 actual fraud transactions.
Model caught 24.
Recall = $24 / 30 = 80\%$

High recall means fewer missed positives.

Important when false negatives are dangerous.

Example:

In disease detection, missing a real disease case can be serious.

Precision vs Recall

Metric	Focus	Useful When
Precision	Avoid false positives	Wrong alerts are costly
Recall	Avoid false negatives	Missing true cases is costly

Example:

Use Case	More Important Metric
Spam detection	Precision
Cancer detection	Recall
Fraud detection	Recall and Precision
Resume shortlisting	Precision
Safety alert system	Recall

F1-Score

F1-score combines precision and recall.

It is useful when we want balance between both.

F1-score = Harmonic mean of Precision and Recall
--

Simple meaning:

F1-score is high only when both precision and recall are reasonably high.

Example:

Model	Precision	Recall	F1-score
Model A	90%	40%	Low/Medium
Model B	70%	75%	Better balanced
Model C	40%	95%	Low/Medium

Model B may be better when we need balanced performance.

ROC-AUC

ROC-AUC measures how well the model separates positive and negative classes.

Simple meaning:

ROC-AUC tells whether the model gives higher risk scores to positive cases than negative cases.

Values:

ROC-AUC	Meaning
0.5	Random guessing
0.7	Acceptable
0.8	Good
0.9+	Excellent

Example:

For churn prediction, ROC-AUC checks whether customers who actually churn usually get higher churn probability scores than customers who do not churn.

ROC-AUC is useful when the model outputs probabilities.

Example:

Customer A churn probability = 0.85
Customer B churn probability = 0.20

The threshold may be changed later based on business needs.

14. Regression Metrics

Even though your list focuses on classification metrics, regression is also important.

MAE: Mean Absolute Error

Simple meaning:

Average absolute mistake.

Example:

If actual house price is ₹80 lakh and predicted price is ₹75 lakh:

Error = ₹5 lakh

MAE tells average error in original unit.

MSE: Mean Squared Error

MSE squares the errors.

It penalizes large errors more.

RMSE: Root Mean Squared Error

RMSE is square root of MSE.

It is also in original unit.

R² Score

R² tells how much variation in target is explained by the model.

R ² Value	Meaning
0	Model is not useful
0.5	Model explains 50% variation
0.8	Good model
1.0	Perfect model

15. Overfitting and Underfitting

Underfitting

Underfitting means the model is too simple.

It performs badly on training data and test data.

Example:

Trying to predict house price using only one feature:

```
number_of_rooms
```

But house price also depends on:

```
location  
area  
age  
floor  
amenities  
nearby facilities
```

So the model cannot learn enough.

Signs of underfitting:

```
Low training accuracy  
Low test accuracy  
Model is too simple  
Important features are missing
```

Overfitting

Overfitting means the model memorizes training data instead of learning general patterns.

It performs very well on training data but poorly on test data.

Example:

A student memorizes answers from practice questions but fails when the exam questions change slightly.

Signs of overfitting:

Very high training accuracy
Low test accuracy
Model is too complex
Model memorizes noise

Comparison table

Concept	Training Performance	Test Performance	Problem
Underfitting	Poor	Poor	Model did not learn enough
Good Fit	Good	Good	Model learned useful patterns
Overfitting	Excellent	Poor	Model memorized training data

ASCII view

Underfitting: Pattern learned is too simple. Actual Pattern: ~~~~~~ Model Prediction: ----- Good Fit: Pattern learned is close to actual behavior. Actual Pattern: ~~~~~~ Model Prediction: ~~~~ ~~~~ Overfitting: Model follows every small noise. Actual Pattern: ~~~~~~ Model Prediction: ~^~v~~~^~

16. How to reduce overfitting

Common methods:

Method	Meaning
More training data	Helps model learn general patterns
Simpler model	Avoid unnecessary complexity
Regularization	Penalizes overly complex models
Cross-validation	Gives more reliable performance estimate
Feature selection	Remove useless or noisy features
Early stopping	Stop training before model memorizes
Data augmentation	Create more training examples

Example:

If a decision tree is overfitting, we can limit its depth.

```
max_depth = 5
```

This prevents the tree from memorizing every small detail.

17. Full Machine Learning Workflow

ASCII diagram showing ML workflow

```

Business Problem
  |
  v
Define ML Problem
  |
  v
Collect Data
  |
  v
Understand Data
  |
  v
Clean Data
  |
  v
Feature Engineering
  |
  v
Train/Test Split
  |
  v
Train Model
  |
  v
Validate Model
  |
  v

```

```
Evaluate Metrics
  |
  v
Tune Model
  |
  v
Select Best Model
  |
  v
Package Model
  |
  v
Deploy as API / Batch Job
  |
  v
Monitor in Production
  |
  v
Retrain When Needed
```

18. Step-by-step ML workflow explanation

Step 1: Understand business problem

Before building a model, understand the actual goal.

Bad problem statement:

```
Build a machine learning model.
```

Good problem statement:

```
Predict which customers are likely to leave in the next 30 days so the retention team can take action.
```

Step 2: Define ML problem

Convert business problem into ML task.

Example:

```
Business problem:
Reduce customer churn.
```

```
ML problem:
Binary classification to predict churn = Yes or No.
```

Step 3: Collect data

Possible data sources:

```
Databases
CSV files
APIs
```



```
Logs
CRM systems
Cloud storage
Data lake
Data warehouse
```

Step 4: Clean data

Common cleaning tasks:

```
Remove duplicates
Handle missing values
Fix incorrect data types
Treat outliers
Standardize categories
```

Step 5: Feature engineering

Create useful variables.

Example:

```
last_login_date -> days_since_last_login
purchase_history -> average_monthly_spend
complaints -> complaint_count_last_90_days
```

Step 6: Split data

Divide into training and testing data.

```
Train data: model learns
Test data: model is evaluated
```

Step 7: Train model

Choose algorithm and train.

Examples:

```
Logistic Regression
Decision Tree
Random Forest
XGBoost
Support Vector Machine
Neural Network
```

Step 8: Evaluate model

Use correct metrics.

For classification:

```
Accuracy
Precision
Recall
```

F1-score ROC-AUC

For regression:

MAE RMSE R^2

Step 9: Tune model

Improve model by adjusting hyperparameters.

Examples:

tree depth learning rate number of estimators regularization strength
--

Step 10: Deploy model

Make the model available to real users or systems.

Deployment options:

REST API Batch scoring job Streaming prediction Embedded inside application Cloud endpoint
--

Step 11: Monitor model

Check if the model continues to work well.

Monitor:

Prediction quality Data drift Model drift Latency Errors Cost Fairness User feedback

Step 12: Retrain model

Retrain when data changes or performance drops.

Example:

Customer behavior changed after new pricing plan.
Old model may no longer work well.
Model needs retraining with latest data.

19. Pseudocode for building an ML model

START

1. Define the business problem
Example: Predict whether a customer will churn
2. Load dataset
data = read customer data
3. Understand dataset
check number of rows and columns
check missing values
check target column distribution
check data types
4. Clean dataset
remove duplicate records
fill missing values
fix incorrect data types
handle outliers if required
5. Create features and target
X = input columns
y = target column
6. Perform feature engineering
create useful new columns
encode categorical columns
scale numerical columns if needed
7. Split data
X_train, X_test, y_train, y_test = train_test_split(X, y)
8. Choose model
model = suitable ML algorithm
Example: Logistic Regression, Random Forest, XGBoost
9. Train model
model.fit(X_train, y_train)
10. Make predictions
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)
11. Evaluate model
calculate accuracy
calculate precision

```
    calculate recall
    calculate F1-score
    calculate ROC-AUC

12. Check overfitting or underfitting
    compare training score and test score

13. Improve model
    tune hyperparameters
    improve features
    try different algorithms
    use cross-validation

14. Select best model
    choose model based on business metric

15. Save model
    save trained model file
    save preprocessing steps

16. Deploy model
    expose model through API or batch pipeline

17. Monitor model
    track predictions, errors, drift, latency, and feedback

18. Retrain when needed

END
```

20. Simple Python-style pseudocode

```
# Step 1: Load data
data = load_dataset("customer_churn.csv")

# Step 2: Separate input and target
X = data.drop("churn", axis=1)
y = data["churn"]

# Step 3: Clean and transform data
X = handle_missing_values(X)
X = encode_categorical_features(X)
X = scale_numeric_features(X)

# Step 4: Split data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
# Step 5: Choose model
model = RandomForestClassifier()

# Step 6: Train model
model.fit(X_train, y_train)

# Step 7: Predict
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[: , 1]

# Step 8: Evaluate
accuracy = calculate_accuracy(y_test, y_pred)
precision = calculate_precision(y_test, y_pred)
recall = calculate_recall(y_test, y_pred)
f1 = calculate_f1_score(y_test, y_pred)
roc_auc = calculate_roc_auc(y_test, y_prob)

# Step 9: Compare results
print(accuracy, precision, recall, f1, roc_auc)

# Step 10: Save model
save_model(model, "churn_model.pkl")

# Step 11: Deploy model
create_api_endpoint(model)

# Step 12: Monitor model in production
monitor_predictions()
monitor_data_drift()
monitor_model_performance()
```

21. Easy end-to-end example

Problem

A telecom company wants to know which customers may leave next month.

ML task

Classification

Target column

churn = Yes / No

Input features

monthly_bill
contract_type
number_of_complaints
internet_usage

```
customer_tenure  
payment_delay_count
```

Model output

```
Customer churn probability = 0.82
```

Business action

```
Retention team gives special offer to high-risk customers.
```

Important metric

Recall may be important because the company does not want to miss customers who are likely to leave.

But precision is also important because giving offers to too many low-risk customers may waste money.

So F1-score or ROC-AUC can be useful.

22. How ML moves from experiment to production

In real companies like IBM, ML is not only about building a model in a notebook.

A production ML system needs to be reliable, repeatable, monitored, and integrated with business systems.

Experiment stage

At this stage, data scientists work in notebooks.

Activities:

```
Explore data  
Try features  
Train different models  
Compare metrics  
Choose best approach
```

Tools may include:

```
Python  
Pandas  
Scikit-learn  
Jupyter Notebook  
Databricks Notebook  
MLflow
```

Production stage

At this stage, the model is used by real applications or users.

Activities:

Package model
Create API
Deploy model
Monitor model
Handle errors
Track model versions
Retrain model

Tools may include:

Docker
REST API
FastAPI
Cloud platform
CI/CD pipeline
MLflow Model Registry
Databricks Jobs
Kubernetes
Monitoring dashboards

Experiment to production flow

```
graph TD
    A[Notebook Experiment] --> B[Reusable Training Script]
    B --> C[Model Tracking]
    C --> D[Model Registry]
    D --> E[Testing]
    E --> F[Deployment]
    F --> G[Monitoring]
    G --> H[Retraining Pipeline]
```

Example

In notebook:

Train churn prediction model and get F1-score = 0.82

In production:

Every night, a batch job scores all active customers.
High-risk customers are sent to CRM system.
Retention team takes action.
Model performance is monitored weekly.

23. Important production concepts

Model versioning

Track which version of the model is used.

Example:

```
churn_model_v1  
churn_model_v2  
churn_model_v3
```

If a new model performs badly, we can roll back to an older version.

Data drift

Data drift means production data changes compared to training data.

Example:

The model was trained before a new pricing plan. After pricing changes, customer behavior changes.

The model may become less accurate.

Model drift

Model drift means the relationship between input and output changes.

Example:

Earlier, high bill amount caused churn. Now, poor network quality may be the bigger reason.

Monitoring

Monitor:

```
Input data distribution  
Prediction distribution  
Accuracy if actual labels are available  
Latency  
API errors  
Business impact
```

Retraining

Retraining means training the model again using newer data.

Example:

Retrain churn model every month using latest customer behavior data.

24. Common mistakes beginners make

Mistake	Why It Is a Problem	Better Approach
Using accuracy for every problem	Misleading for imbalanced data	Use precision, recall, F1, ROC-AUC
Training and testing on same data	Gives fake high performance	Use train/test split
Ignoring missing values	Model may fail or learn wrong patterns	Clean data properly
Not checking target imbalance	Minority class may be ignored	Check class distribution
Doing feature engineering after split incorrectly	Can cause data leakage	Fit transformations only on training data
Choosing complex model first	Harder to explain and debug	Start with simple baseline
Ignoring business goal	Metric may not match real need	Select metric based on business impact
Not saving preprocessing logic	Production predictions may be wrong	Save full pipeline
Not monitoring model	Model may degrade silently	Track drift and performance
Confusing correlation with causation	Pattern may not mean cause	Validate with domain knowledge

25. Very important concept: Data leakage

Data leakage happens when the model gets information during training that would not be available in real life.

Example:

You are predicting whether a customer will churn.

Bad feature:

account_closure_date

Why bad?

Because this information is known only after the customer has already churned.

The model will perform very well in testing but fail in production.

Another example

Problem:

Predict loan default at application time.

Bad feature:

number_of_late_payments_after_loan_approval

This information is from the future and would not be available during prediction.

Always ask:

Would this feature be available at the time of prediction?

If the answer is no, do not use it.

26. Interview-ready explanation

You can explain ML like this in an interview:

Machine Learning is a way to build systems that learn patterns from historical data and use those patterns to make predictions on new unseen data. In supervised learning, we train the model using labeled examples, such as predicting churn or fraud. In unsupervised learning, we do not have labels, so the model finds hidden patterns, such as customer segments.

For classification problems, the output is a category, like fraud or not fraud. For regression problems, the output is a continuous number, like price or demand. I would usually split the data into training and testing sets, use cross-validation for reliable evaluation, and choose metrics based on the business goal. For example, in fraud detection, recall is important because missing fraud is costly, while precision is also important to avoid blocking genuine users.

A model is production-ready only when it is packaged, versioned, deployed, monitored, and retrained when performance drops or data changes.

27. Quick revision table

Topic	One-line Meaning
Machine Learning	Learning patterns from data
Supervised Learning	Learning with correct answers
Unsupervised Learning	Finding patterns without answers
Classification	Predicting categories
Regression	Predicting numbers
Clustering	Grouping similar data
Feature Engineering	Creating useful input variables
Train/Test Split	Separating learning data and testing data

Topic	One-line Meaning
Cross-Validation	Testing model reliability using multiple splits
Accuracy	Overall correct predictions
Precision	Correctness among predicted positives
Recall	Ability to find actual positives
F1-score	Balance between precision and recall
ROC-AUC	Ability to separate classes
Overfitting	Memorizing training data
Underfitting	Learning too little
Production ML	Deploying, monitoring, and retraining model

28. What to remember for IBM AI/GenAI preparation

For IBM AI/GenAI roles, do not treat ML as only theory.

You should connect it with practical delivery:

Data understanding Feature engineering Model training Evaluation Experiment tracking Model deployment Monitoring Governance Retraining Business impact

For GenAI projects, the same thinking applies:

Traditional ML	GenAI / RAG Equivalent
Feature engineering	Chunking, metadata, embeddings
Model evaluation	Answer quality, faithfulness, relevance
Train/test split	Evaluation dataset
Model monitoring	Prompt quality, hallucination, latency
Model deployment	LLM app/API deployment
Model drift	Knowledge base drift or data freshness issue
Retraining	Re-indexing, prompt improvement, fine-tuning

So the core idea is:

A Data Scientist should not only build models.

A Data Scientist should build reliable AI solutions that work on real data, solve business problems, and can be maintained in production.